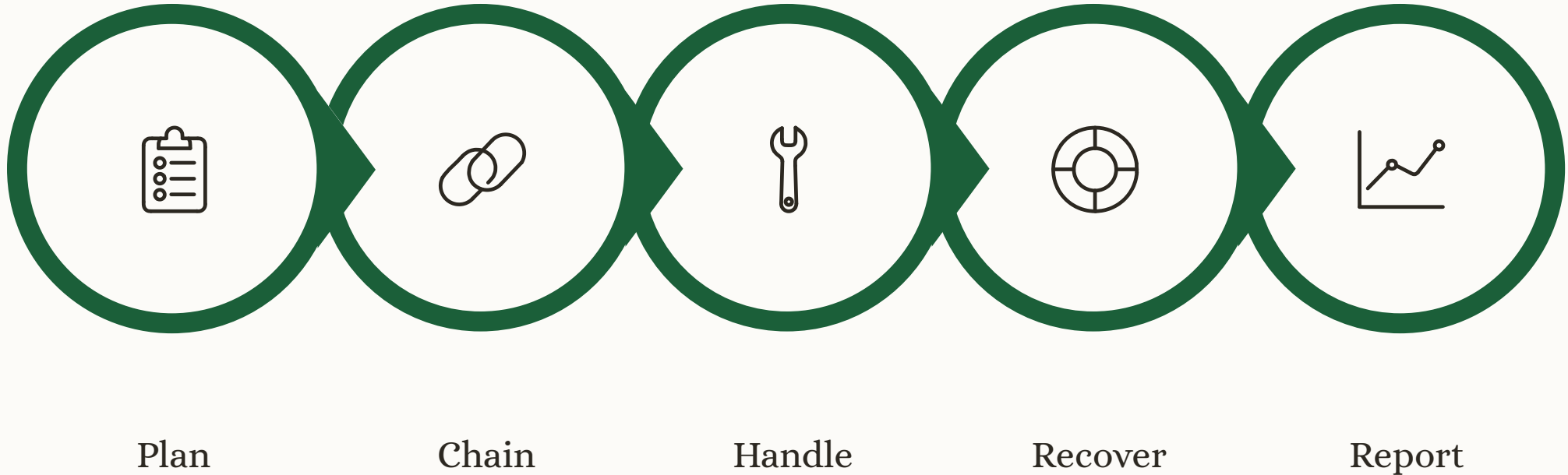


Chaining Promises with `then()` and `catch()`

A structured approach to handling asynchronous operations in JavaScript through sequential Promise chaining and robust error management.



INTRODUCTION

Understanding Promise Chaining

What is Promise Chaining?

Promise chaining is a technique that allows multiple asynchronous operations to execute sequentially, where each operation begins only after the previous one completes successfully. Rather than nesting callbacks (callback hell), chaining creates a linear, readable flow using the `then()` method.

Each `then()` call returns a new Promise, enabling the chain to continue. This maintains clean, maintainable code whilst preserving the asynchronous nature of JavaScript operations.

Why Chaining is Required

Asynchronous operations often depend on results from previous operations. Consider fetching user data, then fetching their posts, then fetching comments—each step requires the previous result.

Without chaining, developers resort to nested callbacks, creating deeply indented, difficult-to-maintain code. Promise chaining solves this by flattening the structure whilst maintaining execution order, improving code readability and error handling capabilities.



CORE CONCEPT

Using then() in Promise Chains

The then() method is the cornerstone of Promise chaining. It accepts callback functions and automatically returns a new Promise, enabling continuous chaining. When a then() callback returns a value, that value is wrapped in a resolved Promise and passed to the next then() in the chain.

01	02	03
then() Returns New Promise	Sequential Execution	Value Propagation
Every then() call creates and returns a fresh Promise object, regardless of what the callback returns. This enables indefinite chaining.	Operations execute in strict order—the second then() waits for the first to complete. The return value becomes the resolved value for the next Promise.	Return values automatically propagate down the chain. Returning a Promise causes the chain to wait for its resolution before continuing.

Example: Sequential Asynchronous Operations

```

fetchUserData(userId)
  .then(user => {
    console.log('User fetched:', user.name);
    return fetchUserPosts(user.id);
  })
  .then(posts => {
    console.log('Posts fetched:', posts.length);
    return fetchPostComments(posts[0].id);
  })
  .then(comments => {
    console.log('Comments fetched:', comments.length);
  });

```

In this example, each then() waits for the previous operation to complete. The return value from each callback becomes the input to the next then(), creating a clear, sequential flow of data through asynchronous operations.



ERROR MANAGEMENT

Error Handling with catch()

Purpose and Mechanism

The `catch()` method handles rejected Promises anywhere in the chain. When any Promise in the chain rejects or throws an error, execution skips all subsequent `then()` blocks and jumps directly to the nearest `catch()`.

This centralised error handling eliminates the need for error callbacks in every `then()` block, simplifying code and ensuring no error goes unhandled. A single `catch()` at the end can handle errors from any point in the chain.

Error Propagation Behaviour

Errors propagate down the chain automatically. If a Promise rejects or a `then()` callback throws an error, subsequent `then()` blocks are skipped until a `catch()` is encountered. This "waterfall" effect ensures errors are always handled, preventing silent failures.

Rejection Handling Example

```
fetchUserData(userId)
  .then(user => {
    if (!user) {
      throw new Error('User not found');
    }
    return fetchUserPosts(user.id);
  })
  .then(posts => {
    return processPostData(posts);
  })
  .catch(error => {
    console.error('Error occurred:', error.message);
    // Handle error appropriately
  });
```

Here, errors from any stage—network failures, validation errors, or processing errors—are caught by the single `catch()` block, providing robust error management.



SUMMARY

Key Concepts and Learning Outcomes

Promise Chaining Flow

Promises chain via `then()`, with each returning a new Promise. Values propagate sequentially, enabling clean asynchronous workflows without callback nesting.

Error Handling Strategy

The `catch()` method provides centralised error management. Errors propagate down the chain until caught, skipping intermediate `then()` blocks automatically.

Advantages Over Callbacks

Chaining eliminates callback hell, improves readability, enables better error handling, and maintains flat code structure whilst preserving asynchronous execution order.

Examination-Relevant Takeaways

- **Return values matter:** Always return Promises from `then()` callbacks to maintain the chain; forgotten returns break sequential execution
- **Error propagation:** Understand that a single `catch()` can handle errors from anywhere upstream in the chain
- **Execution order:** `then()` blocks execute sequentially, not in parallel—each waits for the previous Promise to resolve
- **Best practice:** Place `catch()` at the end of chains for comprehensive error handling; multiple `catch()` blocks can be used for granular error management
- **Comparison advantage:** Promise chains solve "pyramid of doom" (deeply nested callbacks) whilst maintaining clear, linear code structure

